

Agenda at a glance

1. **Deep Math** behind Optimizers
2. How to choose between ***different Loss Functions*** ?

The Mindset?

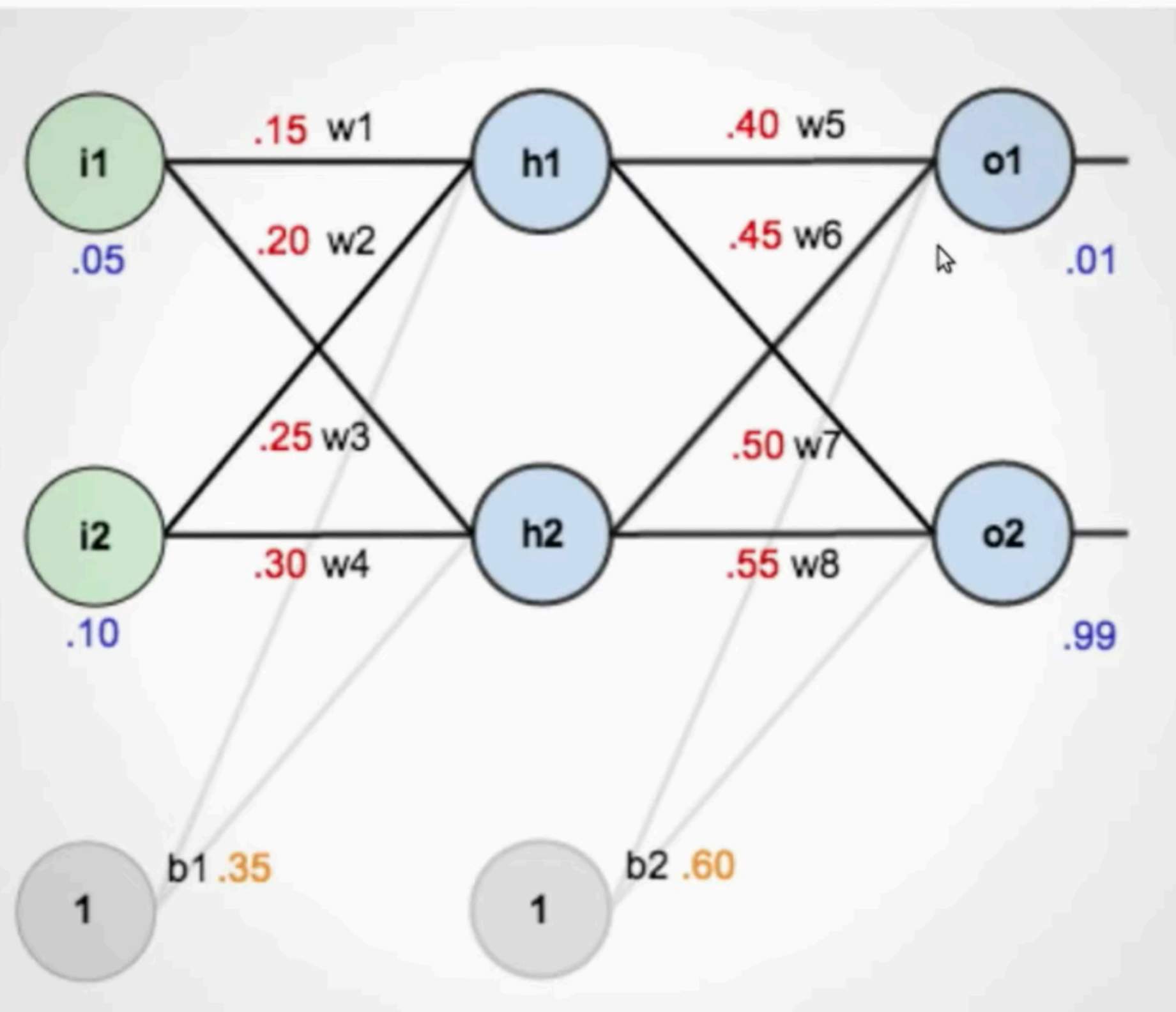
How to *approach this Chapter?* Like Kayaking



Before getting into ***Deep Math behind Optimizers,***
let's do Quick Recap

	A	B	C	D	E	F	G	H
1								
2								
3								
4								
5								
6								
7								
8								
9								
10								
11								
12								
13								
14								
15								
16								
17								
18								
19								
20								
21								
22								
23								
24								
25								
26								
27								
28								
29								
30								
31								
32								
33								
34								
35								
36								
37								
38								
39								
40								
41								
42								
43								
44								
45								
46								
47								
48								
49								
50								





$$i1=0.05, i2=0.10$$

$$w1=0.15, w2=0.20$$

$$w3=0.25, w4=0.30$$

$$b1=0.3$$

$$w5=0.4, w6=0.45$$

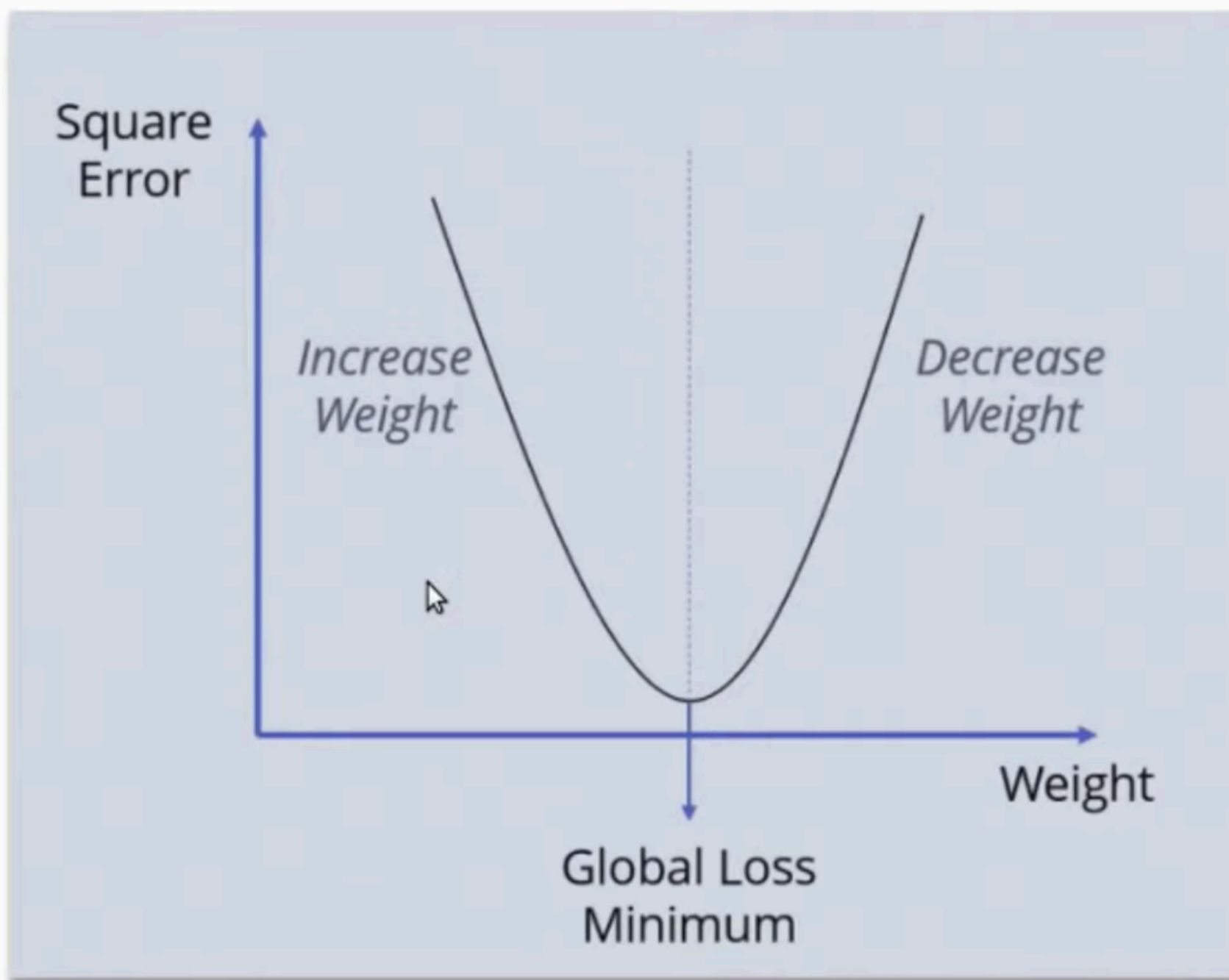
$$w7=0.5, w8=0.55$$

$$b2=0.6$$

$$o1=0.01, o2=0.99$$

GRADIENT DESCENT

- Update the weights using gradient descent.
- Gradient descent is used for finding the minimum of a function.
- In our case we want to minimize the error function.



We can now update w_1 :

$$w_1^+ = w_1 - \eta * \frac{\partial E_{total}}{\partial w_1} = 0.15 - 0.5 * 0.000438568 = 0.149780716$$

Math Equation of
Gradient Descent optimizer

Repeating this for w_2 , w_3 , and w_4

$$w_2^+ = 0.19956143$$

$$w_3^+ = 0.24975114$$

$$w_4^+ = 0.29950229$$

Topic 1

Deep Math behind Optimizers

Optimizers

① Gradient Descent:

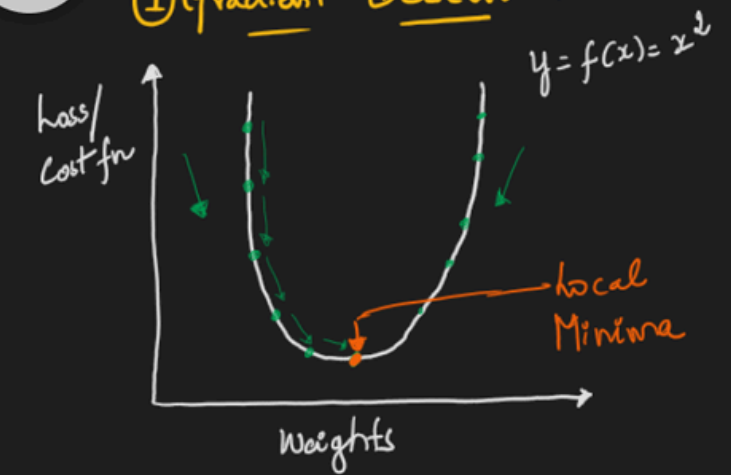


Fig. 2D Representation.

Weight Updation:

$$w_{new} = w_{old} - \alpha_t \left[\frac{\partial L}{\partial w} \right]_{w_{old}}$$

$$bias_{new} = bias_{old} - \alpha_t \left[\frac{\partial L}{\partial b} \right]_{b_{old}}$$

where $\alpha \rightarrow$ LR Rate
which determines
how fast/slow it need to
converge to local minima.

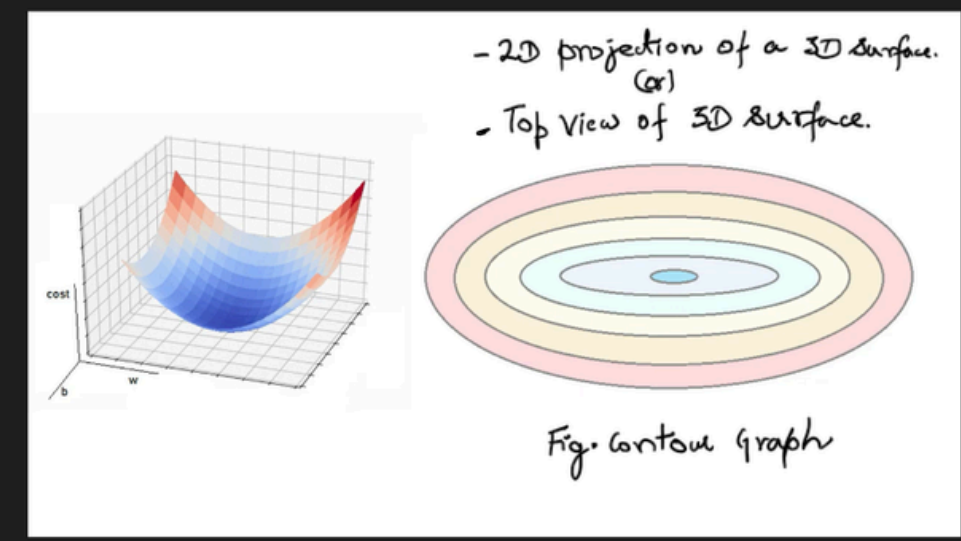
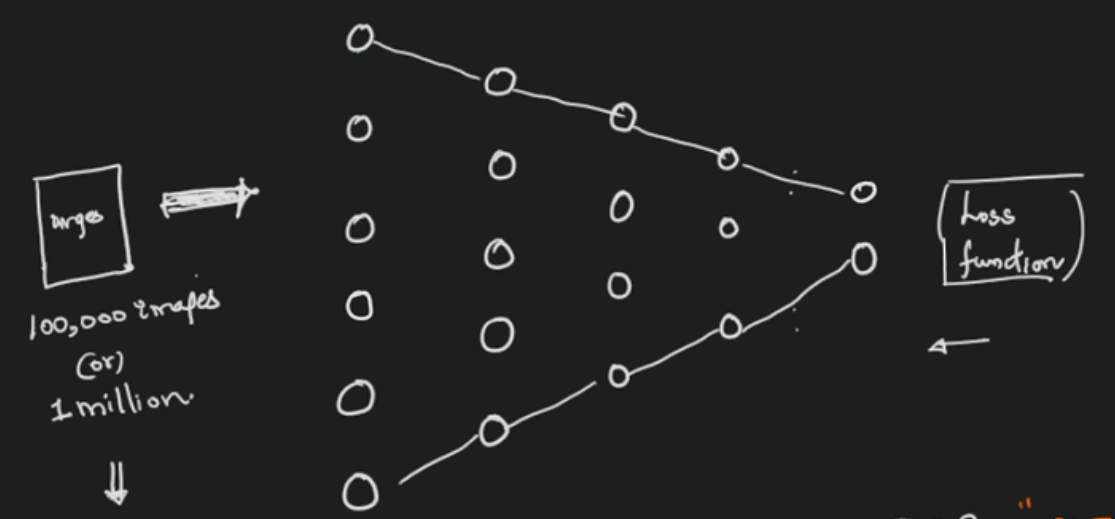


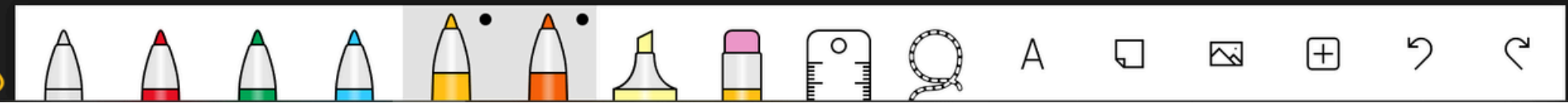
Fig. Contour Graph

I/P H1 H2 H3 O/P



$100000 \times 1000 \times 1000 = 10^{11} = 100 \text{ TB} \Rightarrow \text{RAM/GPU ??? "OUT OF MEMORY ERROR"}$

④ SGD





100000 * 1000 * 1000 = 10¹¹ = 100 TB ⇒ RAM/GPU ??? "OUT OF MEMORY ERROR"

④ SGD with Momentum:

Normal GD Weight Updation eqn is,

$$w_{new} = w_{old} - \alpha_t \cdot \left[\frac{\partial L}{\partial w} \right]_{w_{old}}$$
$$bias_{new} = bias_{old} - \alpha_t \cdot \left[\frac{\partial L}{\partial b} \right]_{b_{old}}$$

SGD with Momentum

$$w_{new} = w_{old} - \alpha_t \cdot \underline{Vdw}$$
$$bias_{new} = bias_{old} - \alpha_t \cdot \underline{Vdb}$$

→ Exponential weighted Moving Average:

$$Vdw = \beta * Vdw_{t-1} + (1-\beta) \frac{\partial L}{\partial w_t}$$
$$Vdb = \beta * Vdb_{t-1} + (1-\beta) \frac{\partial L}{\partial b_t}$$

momentum.

Loss wrt current weight

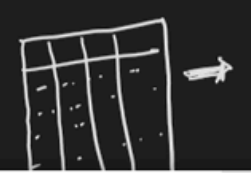
Mov.Avg till previous time

$$0 < \beta < 1$$

Exponential Weighted Moving Avg can be calculated as.

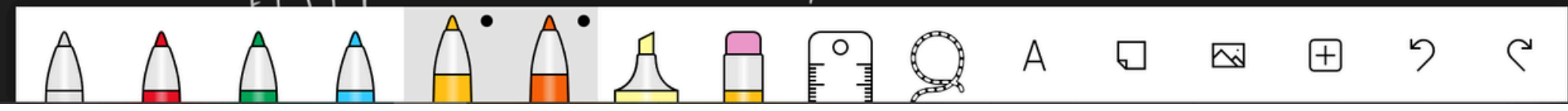
$$V_t = \beta * V_{t-1} + (1-\beta) * \theta_t$$

↑ ↑ ↑
Avg. at time t Avg till previous time New weight.



Now, Let's say $V_0 = 0$,

$$V_1 = \beta * V_0 + (1-\beta) * \theta_1 = (1-\beta) \theta_1 \rightarrow ①$$



$$- \beta * \theta_3 \quad \left| \quad V_4 = \beta * \dots \right.$$

~~new = bias_{dd} - d_t [$\frac{\partial L}{\partial b}$]_{old}~~

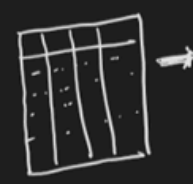
$$Vdw_t = \beta * Vdw_{t-1} + (1-\beta) \frac{\partial L}{\partial w_t} \Rightarrow \beta \text{ momentum.}$$

$$Vdb_t = \beta * Vdb_{t-1} + (1-\beta) \frac{\partial L}{\partial b_t} \quad | 0 < \beta < 1 |$$

Exponential Weighted Moving Avg can be calculated as.

$$V_t = \beta * V_{t-1} + (1-\beta) * \theta_t$$

↑
Avg. at time t
↑
Avg till previous time
↑
New weight.



Now, let's say $V_0 = 0$,

$$V_1 = \beta * V_0 + (1-\beta) * \theta_1 = (1-\beta) \theta_1 \rightarrow (1)$$

$$V_2 = \beta * V_1 + (1-\beta) * \theta_2$$

$$= \beta * (1-\beta) \theta_1 + (1-\beta) * \theta_2$$

$$= (1-\beta) [\beta \theta_1 + \theta_2]$$

$$V_2 = (1-\beta) [\theta_2 + \beta \theta_1] \rightarrow (2)$$

$$V_3 = \beta * V_2 + (1-\beta) * \theta_3$$

$$= \beta * \{ (1-\beta) [\theta_2 + \beta \theta_1] \} + (1-\beta) * \theta_3$$

$$= (1-\beta) [\theta_3 + \beta \theta_2 + \beta^2 \theta_1]$$

$$V_4 = \beta * V_3 + (1-\beta) \theta_4$$

$$= \beta * \{ (1-\beta) [\theta_3 + \beta \theta_2 + \beta^2 \theta_1] \} + (1-\beta) \theta_4$$

$$= (1-\beta) \{ \theta_4 + \beta \theta_3 + \beta^2 \theta_2 + \beta^3 \theta_1 \}$$

If $\beta = 0.9, \Rightarrow \beta^1 > \beta^2 > \beta^3 > \beta^4 \dots$

For t iterations,

$$V_t = (1-\beta) \{ \theta_t + \beta \theta_{t-1} + \beta^2 \theta_{t-2} + \beta^3 \theta_{t-3} + \dots + \beta^t \theta_0 \}$$

ad:

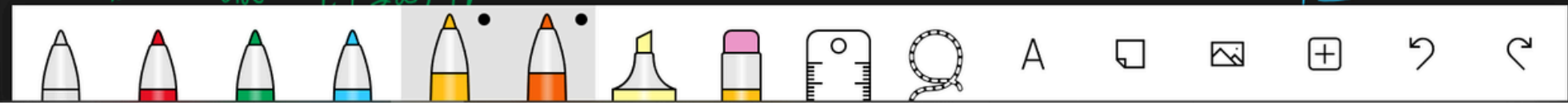
eqn:

$$w_{new} = w_{old} - \alpha \left[\frac{\partial L}{\partial w} \right]_{w_{old}}$$

Adagrad Weight Update:

$$w_{new} = w_{old} - \eta' \left[\frac{\partial L}{\partial w} \right]$$

$$\beta^t \approx 0$$





⑥ Adagrad :

GD eqn :

$$w_{new} = w_{old} - \alpha \left[\frac{\partial L}{\partial w} \right]_{w_{old}}$$

$$b_{new} = b_{old} - \alpha \left[\frac{\partial L}{\partial b} \right]_{b_{old}}$$

Note: α = constant.

Adagrad Weight Updation:

$$w_{new} = w_{old} - \eta' \left[\frac{\partial L}{\partial w} \right]_{old}$$

where $\eta' \rightarrow$ Current L.R.

$$\eta'_t = \frac{\eta}{\sqrt{\alpha_t + \epsilon}}, \text{ where } \eta \Rightarrow \text{Initial LR. } \boxed{\eta = 0.001}$$

where $\alpha_t \rightarrow$ Sum of squared gradients

$$\alpha_t = \sum_{i=1}^t \left(\frac{\partial L}{\partial w_i} \right)^2$$

OBSERVATIONS

After several iterations,

$$\alpha_t \uparrow \uparrow \uparrow \uparrow$$



$$\eta'_t \downarrow \downarrow \downarrow \downarrow$$

[Current Learning Rate]



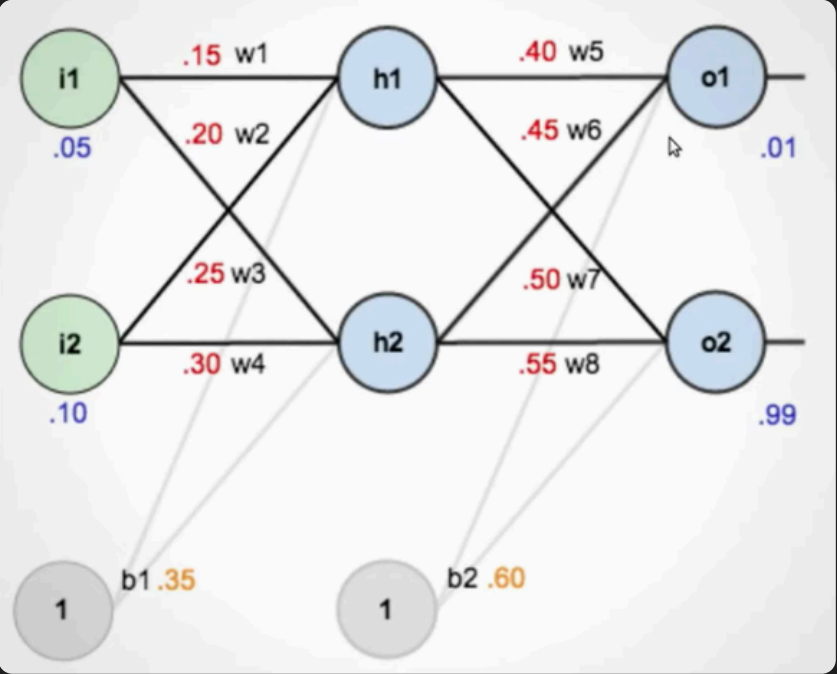
then $\boxed{\text{New Weight} = \text{Old Weight}}$

"Aggressive decay in the learning stage of the gradients"

for t iterations,

$$V_t = (1 - \beta) \nabla \theta_t + \beta \nabla \theta_{t-1} + \beta^2 \nabla \theta_{t-2} + \beta^3 \nabla \theta_{t-3} + \dots$$

$\beta^t \approx 0$





⑦ RMS Prop:

Adagrad Weight Update:

$$w_{\text{new}} = w_{\text{old}} - \overset{\text{the gradient}}{\eta'_t} \left[\frac{\partial L}{\partial w} \right]_{\text{old}} \quad (\text{dw})$$

Small change in the denominator $\rightarrow \eta'_t = \frac{\eta}{\sqrt{w_{\text{avg}} + \epsilon}}$ exp. weigh. Mov. Avg.

where $w_{\text{avg}} = \gamma * w_{\text{avg}, t-1} + (1-\gamma) * \left(\frac{\partial L}{\partial w} \right)^2_{\text{all other gradients}}$

$\gamma = 0.9$

$$w_{\text{avg}, t} = 0.9 * w_{\text{avg}, t-1} + 0.1 * \left(\frac{\partial L}{\partial w} \right)^2$$

90% weightage to the previous datapoints

10% weightage to the sum of all other squared gradients.

⑧ ADAM



⑧ ADAM

"Combination of Momentum Optimizer & RMS prop Optimizer".

Momentum Weight Update:

$$w = w - \alpha \cdot Vdw$$

$$b = b - \alpha \cdot Vdb$$

where, $Vdw = \beta_1 \cdot Vdw_{prev} + (1 - \beta_1) dw$
 $Vdb = \beta_1 \cdot Vdb_{prev} + (1 - \beta_1) db$

Mov.Avg till previous time

Loss wrt current weight

RMS Prop Optimizer

$$w = w - \alpha \cdot \frac{dw}{\sqrt{w_{avg} + \epsilon}}$$

$$b = b - \alpha \cdot \frac{db}{\sqrt{b_{avg} + \epsilon}}$$

where, $w_{avg}_t = \beta_2 \cdot w_{avg}_{prev} + (1 - \beta_2) (dw)^2$

$$b_{avg}_t = \beta_2 \cdot b_{avg}_{prev} + (1 - \beta_2) (db)^2$$

90% weightage to the one step behind datapoints & **10%** to all other gradients

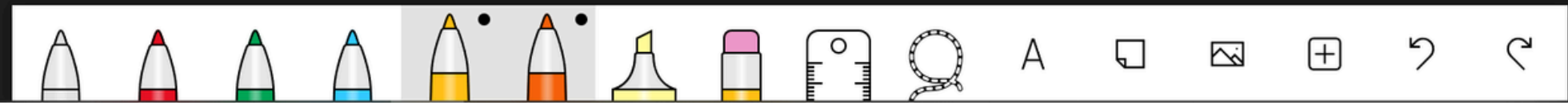
Adam Weight Update

$$w = w - \alpha \cdot \frac{-Vdw}{\sqrt{w_{avg} + \epsilon}}$$

$$b = b - \alpha \cdot \frac{Vdb}{\sqrt{b_{avg} + \epsilon}}$$

$\beta_1 = 0.9$

$\beta_2 = 0.99$



Prompt it...

How to Prompt Chatgpt for better clarity

My Notes

Topic 2

How to choose between ***different Loss Functions***
which have already seen?

How to choose between **different Loss Functions** which have already seen?

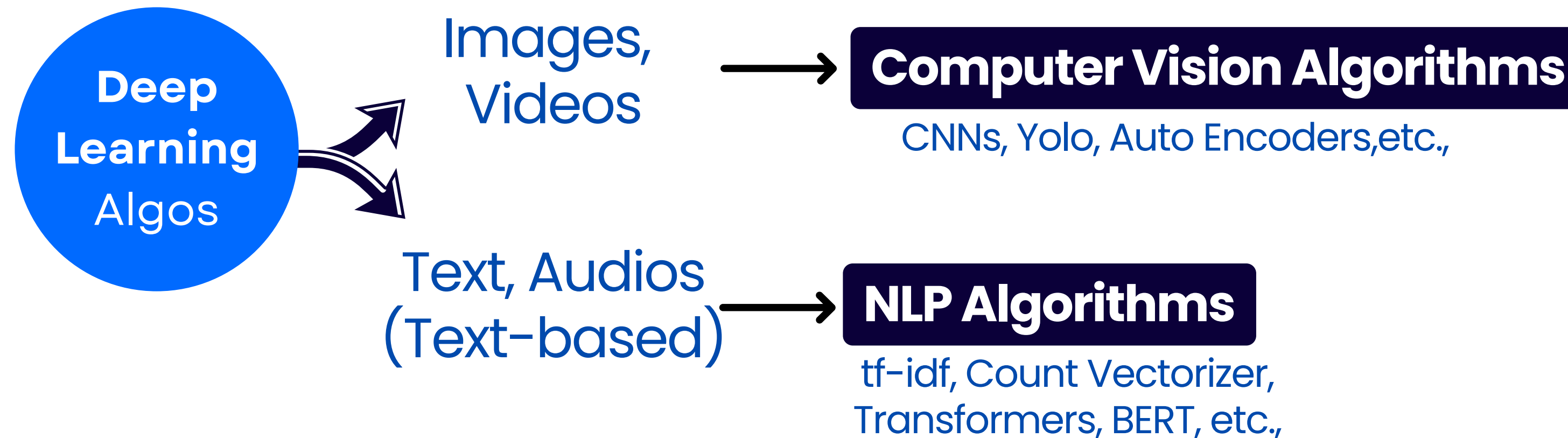
Problem Type	Output Type	Output Layer – Activation Function	Loss Function
Classification	Binary value (0 or 1)	sigmoid	binary_crossentropy
Classification	Single label, multiple classes	softmax	categorical_crossentropy
Classification	Multiple labels, multiple classes	sigmoid	binary_crossentropy
Regression	Numerical value (0-1)	sigmoid	mse or binary_crossentropy
Regression	Numerical value	linear (default)	mse

For More Details...
Refer Documentation...

The Next Chapter...

The Most Waited ***Scripting Part...***

1 Point to be Noted...



Then for **Structured Data?**

"Let's build a ***Deep Neural Network***"